# Mobile Application Development (23IT703)

Experiment-wise program separation extracted from MADRecord_new.pdf.

Note: Code is transcribed from the supplied record. The PDF contains some incomplete or malformed code in a few experiments; those parts are flagged and have not been silently corrected.

## Experiment 1 – Study and Installation of Flutter/Kotlin Multi-platform Environment

```
No standalone program is given in the record.


Flutter commands:
flutter create my_flutter_project
cd my_flutter_project
flutter run


Kotlin Multiplatform:
The record describes creating a Kotlin Multiplatform project through IntelliJ IDEA and
configuring shared and platform-specific code. No source-code listing is provided.
```

## Experiment 2 – Widgets, GUI Components, Fonts and Colors

```
Language: Kotlin (JavaFX)

import javafx.application.Application
import javafx.geometry.Insets
import javafx.scene.Scene
import javafx.scene.control.Button
import javafx.scene.control.ComboBox
import javafx.scene.control.Label
import javafx.scene.control.TextField
import javafx.scene.layout.VBox
import javafx.scene.text.Font
import javafx.stage.Stage

class MyApp: Application() {
    override fun start(primaryStage: Stage) {
        primaryStage.title = "Widget App"

        val root = VBox(10.0)
        root.padding = Insets(10.0)

        // Set a custom font
        val customFont = Font("Arial", 12.0)
```

```
        // Set background color
        root.style = "-fx-background-color: #f0f0f0;"

        // Create a label
        val label = Label("Welcome to Widget App")
        label.font = customFont
        root.children.add(label)

        // Create a button
        val button = Button("Click Me!")
        button.font = customFont
        button.setOnAction { onButtonClick() }
        root.children.add(button)

        // Create a text field
        val textField = TextField()
        textField.font = customFont
        root.children.add(textField)

        // Create a combo box
        val comboBoxValues = listOf("Option 1", "Option 2", "Option 3")
        val comboBox = ComboBox<String>()
        comboBox.items.addAll(comboBoxValues)
        comboBox.font = customFont
        root.children.add(comboBox)

        val scene = Scene(root, 300.0, 200.0)
        primaryStage.scene = scene
        primaryStage.show()
    }

    private fun onButtonClick() {
        println("Button Clicked!")
    }
}

fun main() {
    Application.launch(MyApp::class.java)
}
```

## Experiment 3 – Develop a Native Calculator Application

Language: XML + Kotlin

XML (activity_main.xml):

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

```
    <EditText
        android:id="@+id/inputText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:layout_marginBottom="16dp"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:hint="Enter expression"
        android:inputType="none"
        android:textAlignment="center" />

    <GridLayout
        android:id="@+id/gridLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/inputText"
        android:layout_marginTop="16dp"
        android:columnCount="4"
        android:orientation="horizontal">
        <!-- Buttons for digits and operators go here -->
    </GridLayout>
</RelativeLayout>

Kotlin (MainActivity.kt):
import android.os.Bundle
import android.view.View
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity
import net.objecthunter.exp4j.ExpressionBuilder

class MainActivity : AppCompatActivity() {
    private lateinit var inputText: EditText

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        inputText = findViewById(R.id.inputText)

        val buttons = intArrayOf(
            R.id.btno, R.id.btn1, R.id.btn2, R.id.btn3, R.id.btn4,
            R.id.btn5, R.id.btn6, R.id.btn7, R.id.btn8, R.id.btn9,
            R.id.btnAdd, R.id.btnSub, R.id.btnMul, R.id.btnDiv,
            R.id.btnDot, R.id.btnEquals
        )

        for (buttonId in buttons) {
            val button = findViewById<Button>(buttonId)
            button.setOnClickListener { onButtonClick(it) }
        }
    }
```

```
    private fun onButtonClick(view: View) {
        val currentText = inputText.text.toString()
        val buttonText = (view as Button).text.toString()

        when (view.id) {
            R.id.btnEquals -> evaluateExpression(currentText)
            R.id.btnDot -> handleDotButtonClick(currentText)
            else -> inputText.setText("$currentText$buttonText")
        }
    }

    private fun evaluateExpression(expression: String) {
        try {
            val result = ExpressionBuilder(expression).build().evaluate()
            inputText.setText(result.toString())
        } catch (e: Exception) {
            inputText.setText("Error")
        }
    }

    private fun handleDotButtonClick(currentText: String) {
        val lastOperatorIndex = getLastOperatorIndex(currentText)
        val lastDotIndex = currentText.lastIndexOf(".")

        if (lastDotIndex > lastOperatorIndex) {
            // If there is already a dot after the last operator, do nothing.
            return
        }
        inputText.setText("$currentText.")
    }

    private fun getLastOperatorIndex(text: String): Int {
        val operators = charArrayOf('+', '-', '*', '/')
        var lastOperatorIndex = -1

        for (operator in operators) {
            val index = text.lastIndexOf(operator)
            if (index > lastOperatorIndex) {
                lastOperatorIndex = index
            }
        }
        return lastOperatorIndex
    }
}
```

PDF note: The XML GridLayout contains no button definitions, although the Kotlin code refers to button IDs. The record also has a possible typo in the zero-button ID (btno).

## Experiment 4 – Develop a Gaming Application Using 2D Animations and Gestures

Language: Kotlin (LibGDX)

Main Game Class (transcribed with obvious PDF formatting errors normalized only where unambiguous):

```
import com.badlogic.gdx.ApplicationAdapter
import com.badlogic.gdx.Gdx
import com.badlogic.gdx.graphics.GL20
import com.badlogic.gdx.graphics.Texture
import com.badlogic.gdx.graphics.g2d.SpriteBatch

class GestureQuest : ApplicationAdapter() {
    private lateinit var batch: SpriteBatch
    private lateinit var img: Texture

    override fun create() {
        batch = SpriteBatch()
        img = Texture("badlogic.jpg") // Replace with your own image file
    }

    override fun render() {
        Gdx.gl.glClearColor(1f, 1f, 1f, 1f)
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)
        batch.begin()
        batch.draw(img, 0f, 0f)
        batch.end()
    }

    override fun dispose() {
        batch.dispose()
        img.dispose()
    }
}
```

Gesture Recognition (PDF contains malformed method/type names):

```
import com.badlogic.gdx.InputProcessor

class MyInputProcessor : InputProcessor {
    override fun touchDown(screenX: Int, screenY: Int, pointer: Int, button: Int): Boolean {
        // Handle touch down event
        return true
    }

    override fun keyDown(keycode: Int): Boolean {
        // Handle key down event
        return false
    }

    // Implement other InputProcessor methods as needed
    override fun touchUp(screenX: Int, screenY: Int, pointer: Int, button: Int) {}
    override fun keyUp(keycode: Int) {}
    override fun keyTyped(character: Char) = false
    override fun touchDragged(screenX: Int, screenY: Int, pointer: Int) {}
    override fun mouseMoved(screenX: Int, screenY: Int) = false
    override fun scrolled(amountX: Float, amountY: Float) = false
}
```

Add this in the main game class create() method:
val inputProcessor = MyInputProcessor()
Gdx.input.inputProcessor = inputProcessor

Note: The original PDF has several corrupted tokens in this experiment. The normalized signatures above may need to match the LibGDX version used by your project.

## Experiment 5 – Develop a Movie Rating Application (Similar to IMDb)

Language: Kotlin

```
data class Movie(val title: String, val genre: String, val releaseYear: Int)
data class Rating(val user: String, val movie: String, val rating: Double)

class MovieRatingSystem {
    private val movies = mutableListOf<Movie>()
    private val ratings = mutableListOf<Rating>()

    fun addMovie(movie: Movie) {
        movies.add(movie)
    }

    fun rateMovie(user: String, movieTitle: String, rating: Double) {
        val movie = movies.find { it.title == movieTitle }

        if (movie != null) {
            val userRating = Rating(user, movieTitle, rating)
            ratings.add(userRating)
            println("Rating added successfully.")
        } else {
            println("Movie not found.")
        }
    }

    fun getAverageRating(movieTitle: String): Double {
        val movieRatings = ratings.filter { it.movie == movieTitle }
        return if (movieRatings.isNotEmpty()) {
            val totalRating = movieRatings.map { it.rating }.sum()
            totalRating / movieRatings.size
        } else {
            0.0
        }
    }

    fun listMovies() {
        movies.forEach {
            println("${it.title} (${it.releaseYear}) - ${it.genre}")
        }
    }
}
```

```
fun main() {
    val movieRatingSystem = MovieRatingSystem()

    // Adding movies
    movieRatingSystem.addMovie(Movie("Inception", "Sci-Fi", 2010))
    movieRatingSystem.addMovie(Movie("The Shawshank Redemption", "Drama", 1994))
    movieRatingSystem.addMovie(Movie("The Dark Knight", "Action", 2008))

    // Listing movies
    println("Available Movies:")
    movieRatingSystem.listMovies()

    // Rating movies
    movieRatingSystem.rateMovie("user1", "Inception", 9.0)
    movieRatingSystem.rateMovie("user2", "Inception", 8.5)
    movieRatingSystem.rateMovie("user1", "Black Widow", 9.5)

    // Getting average ratings
    println("Average Rating for Inception:
${movieRatingSystem.getAverageRating("Inception")}")
    println("Average Rating for The Shawshank Redemption:
${movieRatingSystem.getAverageRating("The Shawshank Redemption")}")
    println("Average Rating for The Dark Knight: ${movieRatingSystem.getAverageRating("The
Dark Knight")}")
}
```

Note: The record's procedure describes a Spring Boot application, but the supplied program is a simple Kotlin console program.

## Experiment 6 – Connect to a Web Service and Retrieve Data with HTTP

Language: Kotlin (Ktor client; API URL in record is a placeholder)

```
import io.ktor.client.HttpClient
import io.ktor.client.engine.apache.Apache
import io.ktor.client.request.get

suspend fun fetchData(url: String): String {
    val client = HttpClient(Apache)
    return try {
        val response = client.get<String>(url)
        response
    } finally {
        client.close()
    }
}

fun main() {
    val apiUrl = "https://api.example.com/data"

    try {
        val result = fetchData(apiUrl)
        println("Data received: $result")
```

```
    } catch (e: Exception) {
        println("Error: ${e.message}")
    }
}
```

Note: The record uses a placeholder API URL and calls a suspend function directly from main; this may require changes to run.

## Experiment 7 – Develop a Simple Shopping Application

Language: Kotlin

```
import java.util.Scanner

data class Product(val id: Int, val name: String, val price: Double)

fun displayProducts(products: List<Product>) {
    println("\nAvailable Products:")
    for (product in products) {
        println("${product.id}. ${product.name} - $${product.price}")
    }
}

fun addToCart(cart: MutableList<Product>, product: Product) {
    cart.add(product)
    println("${product.name} added to the cart.")
}

fun viewCart(cart: List<Product>) {
    if (cart.isEmpty()) {
        println("Your cart is empty.")
    } else {
        println("Your Cart:")
        for ((index, item) in cart.withIndex()) {
            println("${index + 1}. ${item.name} - $${item.price}")
        }

        val totalPrice = cart.sumOf { it.price }
        println("Total: $$totalPrice")
    }
}

fun main() {
    val scanner = Scanner(System.`in`)
    val cart = mutableListOf<Product>()
    val products = listOf(
        Product(1, "Product A", 10.99),
        Product(2, "Product B", 19.99),
        Product(3, "Product C", 7.49)
    )

    println("Simple Shopping Cart!")
```

```
    while (true) {
        println("\nMenu:")
        println("1. View Products")
        println("2. Add to Cart")
        println("3. View Cart")
        println("4. Exit")
        print("Enter your choice: ")

        when (scanner.nextInt()) {
            1 -> displayProducts(products)

            2 -> {
                println("\nEnter the product ID to add to cart:")
                val productId = scanner.nextInt()
                val selectedProduct = products.find { it.id == productId }

                if (selectedProduct != null) {
                    addToCart(cart, selectedProduct)
                } else {
                    println("Invalid product ID. Please try again.")
                }
            }

            3 -> viewCart(cart)

            4 -> {
                println("Thank you for using the Simple Shopping App. Exiting...")
                return
            }

            else -> println("Invalid choice. Please enter a valid option.")
        }
    }
}
```

Note: Formatting/typos in the PDF were normalized in this transcription so the Kotlin structure is readable.

## Experiment 8 – Design a Web Server Supporting Push Notifications

Language: Kotlin (Ktor 1.6.10) + HTML

Gradle dependencies shown in the record:

```
plugins {
    kotlin("jvm") version "1.5.31"
    id("io.ktor") version "1.6.10"
}

repositories {
    mavenCentral()
}
```

```
dependencies {
    implementation("io.ktor:ktor-server-netty:1.6.10")
    implementation("io.ktor:ktor-websockets:1.6.10")
}

Kotlin server (source structure; the PDF has malformed identifiers/imports):
import io.ktor.application.*
import io.ktor.features.ContentNegotiation
import io.ktor.features.StatusPages
import io.ktor.http.HttpStatusCode
import io.ktor.jackson.jackson
import io.ktor.response.respondText
import io.ktor.routing.routing
import io.ktor.server.engine.embeddedServer
import io.ktor.server.netty.Netty
import io.ktor.websocket.*
import kotlinx.coroutines.channels.consumeEach
import java.time.Duration

fun Application.module() {
    install(ContentNegotiation) {
        jackson {}
    }

    install(StatusPages) {
        exception<Throwable> { cause ->
            call.respondText(
                cause.localizedMessage ?: "Server error",
                status = HttpStatusCode.InternalServerError
            )
        }
    }

    install(WebSockets) {
        pingPeriod = Duration.ofSeconds(60)
        timeout = Duration.ofSeconds(15)
        maxFrameSize = Long.MAX_VALUE
        masking = false
    }

    routing {
        val connections = mutableListOf<DefaultWebSocketServerSession>()

        webSocket("/push") {
            connections.add(this)
            try {
                incoming.consumeEach { frame ->
                    if (frame is Frame.Text) {
                        val message = frame.readText()
                        connections.forEach {
                            if (it != this) {
                                it.send(Frame.Text(message))
                            }
                        }
```

```
                    }
                }
            } finally {
                connections.remove(this)
            }
        }
    }
}

fun main() {
    embeddedServer(Netty, port = 8080, module = Application::module).start(wait = true)
}
```

HTML client (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebSocket Push Notification</title>
</head>
<body>
    <input type="text" id="messageInput" placeholder="Type a message">
    <button onclick="sendMessage()">Send Message</button>
    <ul id="messages"></ul>

    <script>
        const socket = new WebSocket("ws://localhost:8080/push");

        socket.onmessage = (event) => {
            const messages = document.getElementById("messages");
            const li = document.createElement("li");
            li.appendChild(document.createTextNode(event.data));
            messages.appendChild(li);
        };

        function sendMessage() {
            const input = document.getElementById("messageInput");
            const message = input.value;
            socket.send(message);
            input.value = "";
        }
    </script>
</body>
</html>
```

Note: The PDF contains malformed imports/types. Verify against the Ktor version configured in your Gradle project.

## Experiment 9 – Develop an Application by Integrating Google Maps

Language: XML + Kotlin

```
AndroidManifest.xml entries:
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

Inside <application>:
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY" />

fragment_map.xml:
<fragment
    android:id="@+id/mapFragment"
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />

Kotlin:
import android.os.Bundle
import android.view.View
import androidx.fragment.app.Fragment
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps.OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment

class MapFragment : Fragment(R.layout.fragment_map), OnMapReadyCallback {
    private lateinit var googleMap: GoogleMap

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val mapFragment = childFragmentManager
            .findFragmentById(R.id.mapFragment) as SupportMapFragment
        mapFragment.getMapAsync(this)
    }

    override fun onMapReady(map: GoogleMap) {
        googleMap = map
        // Add map configurations here, such as map type and markers.
    }
}

Dependency shown in the record:
implementation("com.google.android.gms:play-services-maps:17.0.1")

Note: Replace YOUR_API_KEY with a valid key and enable Maps SDK for Android in Google Cloud.
```

## Experiment 10 – Mini Project: BMI Calculator App

```
Language: Dart (Flutter)

import 'package:flutter/material.dart';
```

```
void main() => runApp(BMICalculator());

class BMICalculator extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: InputPage(),
        );
    }
}

class InputPage extends StatefulWidget {
    @override
    _InputPageState createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text('BMI CALCULATOR'),
            ),
            body: Center(
                child: Text('Body Text'),
            ),
            floatingActionButton: FloatingActionButton(
                onPressed: () {},
                child: Icon(Icons.add),
            ),
        );
    }
}
```

Important: The supplied PDF code only displays "Body Text"; it does not actually calculate BMI.

## Experiment 11 – Real-Time On-Device Object Detection Using Flutter and TensorFlow Lite

Language: Dart (Flutter + TensorFlow Lite)

Dependencies shown in the record (pubspec.yaml):

```
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.10.6
  tflite_flutter: ^0.11.0
```

Asset entries shown in the record:

```
flutter:
  assets:
```

```
    - assets/models/ssd_mobilenet.tflite
    - assets/models/ssd_mobilenet.txt

Model loading snippet:
import 'package:tflite_flutter/tflite_flutter.dart';

late Interpreter interpreter;
List<String> labels = [];

Future<void> loadModel() async {
    final options = InterpreterOptions()..threads = 2;
    interpreter = await Interpreter.fromAsset(
        'assets/models/ssd_mobilenet.tflite',
        options: options
    );
    // Code to parse the ssd_mobilenet.txt lines into the labels list...
}

Inference snippet from the record:
// For an object detection model expecting a [1, 300, 300, 3] tensor
var input = [imageMatrix];

// Define structure for output bounding boxes, classes, confidence scores
var output = {
    0: List.filled(1, List.filled(10, List.filled(4, 0.0))), // Bounding boxes
    1: List.filled(1, List.filled(10, 0.0)),                 // Class IDs
    2: List.filled(1, List.filled(10, 0.0)),                 // Scores
    3: List.filled(1, 0.0)                                   // Total detections
};

interpreter.runForMultipleInputs([input], output);

Note: The record does not provide a complete runnable app. It omits camera-frame conversion,
complete label parsing, and drawing code. It also shows model and label files that must be
supplied separately.
```

# RUN GUIDANCE – HOW TO EXECUTE EACH EXPERIMENT

Use the instructions below after reviewing the matching experiment code. The run steps are practical guidance; where the original record is incomplete, the limitation is stated.

## Experiment 1 – Study and Installation of Flutter/Kotlin Multi-platform Environment

**How to run / verify:**

1. Install Flutter SDK and add its bin folder to Windows PATH.

2. Open a new terminal and run: flutter doctor

3. Fix the missing items reported by flutter doctor. Android Studio should be used to install Android SDK components.

4. To test Flutter, run: flutter create my_flutter_project

5. Run: cd my_flutter_project

6. Run: flutter run -d chrome (requires Chrome) or flutter run after starting an Android emulator.

7. For Kotlin Multiplatform, open IntelliJ IDEA, create a Kotlin Multiplatform project, then build/run the platform target generated by the wizard.

Note: This experiment is primarily installation and environment verification; the record does not provide a separate application program.

## Experiment 2 – Widgets, GUI Components, Fonts and Colors

**How to run (Kotlin + JavaFX desktop app):**

1. Open IntelliJ IDEA and create a Kotlin/JVM Gradle project.

2. Add JavaFX dependencies to the Gradle build file. The record's procedure uses JavaFX SDK 17 and requires the JavaFX controls module.

3. Create MyApp.kt under src/main/kotlin and paste the Kotlin program above.

4. Configure JavaFX for your installed JDK/JavaFX version. If using a downloaded JavaFX SDK, set the VM option --module-path <javafx-sdk-path>/lib --add-modules javafx.controls,javafx.fxml (replace the path with your actual path).

5. Run the main() function in MyApp.kt. A desktop window titled Widget App should appear.

Expected interaction: see a label, a Click Me! button, a text field and a combo box. Clicking the button prints Button Clicked! in the Run console.

*Important: This is a JavaFX desktop program, not an Android app. Do not create it inside the Android Studio Android project.*

## Experiment 3 – Develop a Native Calculator Application

**How to run (Android app):**

1. In Android Studio, create a new project using Empty Views Activity (not Empty Activity/Compose), with Kotlin.

2. Open app/src/main/res/layout/activity_main.xml and add the EditText and calculator buttons inside the GridLayout. The record leaves the GridLayout empty, so the button views must be added and their IDs must match the Kotlin IDs.

3. Open MainActivity.kt and replace the generated activity code with the calculator Kotlin code above.

4. Add the exp4j dependency to the app-level Gradle file, sync Gradle, and make sure the package/imports match your project.

5. Start an emulator from Device Manager or connect an Android phone with USB debugging enabled.

6. Select the device and click Run (green triangle).

*Important: The PDF's XML does not define the buttons required by the Kotlin program, and the zero button ID may be mistyped. It will not run until those IDs and views are made consistent.*

## Experiment 4 – Develop a Gaming Application Using 2D Animations and Gestures

**How to run (LibGDX):**

1. Create a LibGDX project using the LibGDX project generator; select Kotlin/JVM or the project template supported by your course setup, and include a desktop launcher.

2. Put the game class in the core module and the image file (for example badlogic.jpg) in the core assets folder.

3. Add the input processor class and register it in the game's create() method.

4. Generate/import the Gradle project in IntelliJ IDEA and allow Gradle sync to finish.

5. Run the desktop launcher task (often named lwjgl3:run, depending on the generated project).

*Important: The record's original code has malformed method names and incomplete InputProcessor implementation. The code above is a readable outline, not a guaranteed complete project. The exact run task and method signatures depend on the LibGDX template/version.*

## Experiment 5 – Develop a Movie Rating Application (Similar to IMDb)

**How to run (Kotlin console program):**

1. Open IntelliJ IDEA and create a Kotlin/JVM project.

2. Create a file named Main.kt under src/main/kotlin.

3. Paste the Movie, Rating, MovieRatingSystem and main() code above.

4. Click the green Run icon beside main(), or use the Gradle run task if the project is configured with the application plugin.

5. Read the movie list and rating/average results in the Run console.

*Important: The record's procedure mentions Spring Boot, but the supplied program is a standalone Kotlin console program. This code does not start a web server.*

## Experiment 6 – Connect to a Web Service and Retrieve Data with HTTP

**How to run (Ktor client):**

1. Open IntelliJ IDEA and create a Kotlin/JVM Gradle project.

2. Add Ktor client core and the matching HTTP engine dependencies to build.gradle.kts. Keep all Ktor dependencies on the same version.

3. Create a Kotlin file with the HTTP client code above.

4. Replace https://api.example.com/data with a real, reachable HTTP API endpoint that returns data.

5. Because fetchData() is suspend, call it from a coroutine (for example runBlocking in a simple console test) rather than calling it directly from a normal main() function.

6. Run main() and inspect the response or error in the Run console.

*Important: The URL in the record is only a placeholder. The code as printed in the record is not directly runnable without a real endpoint and coroutine handling.*

## Experiment 7 – Develop a Simple Shopping Application

**How to run (Kotlin console program):**

1. Open IntelliJ IDEA and create a Kotlin/JVM project.

2. Create Main.kt under src/main/kotlin and paste the shopping application code above.

3. Run the main() function.

4. In the Run console, enter 1 to view products, 2 to add a product, 3 to view the cart, or 4 to exit.

5. When choosing option 2, enter one of the displayed product IDs.

Expected result: the selected product is added to the cart and option 3 displays the cart and total price.

## Experiment 8 – Design a Web Server Supporting Push Notifications

**How to run (Ktor WebSocket server + HTML client):**

1. Create a Kotlin/JVM Gradle project in IntelliJ IDEA and add the Ktor server Netty, WebSockets and other dependencies using one consistent Ktor version.

2. Put the Kotlin server code in the source folder and run its main() function. It should start on port 8080.

3. Save the HTML client code as index.html.

4. Open index.html in a browser. For more reliable browser access, serve the file through a local development server rather than opening it as file://.

5. Open the same page in two browser tabs. Type a message in one tab and click Send Message; the other connected tab should receive it.

*Important: The record uses Ktor 1.6.10 and has malformed imports/type names. If your project uses a newer Ktor version, the server code must be adapted to that version.*

## Experiment 9 – Develop an Application by Integrating Google Maps

**How to run (Android app):**

1. In Android Studio, create an Android project using Empty Views Activity and Kotlin.

2. Add the Google Maps dependency to the app-level Gradle file and sync Gradle.

3. In Google Cloud Console, create/select a project, enable Maps SDK for Android, and create an API key. Restrict the key appropriately for your app.

4. Add the Internet permission and the API-key meta-data inside the application element of AndroidManifest.xml.

5. Create fragment_map.xml with the SupportMapFragment layout shown above.

6. Add the MapFragment Kotlin code and ensure the fragment is actually hosted by an activity/layout.

7. Run on an emulator or Android phone with internet access. A valid API key is required for map tiles to load.

*Important: The PDF gives only the map fragment portion; it does not include a complete host activity or API key.*

## Experiment 10 – Mini Project: BMI Calculator App

**How to run (Flutter):**

1. Install Flutter SDK and the Flutter/Dart extensions in VS Code.

2. Open a terminal and run: flutter doctor

3. Create the project: flutter create bmical

4. Open the bmical folder in VS Code.

5. Replace lib/main.dart with the Flutter code above and save.

6. Run flutter pub get, then run flutter run -d chrome (or start an Android emulator and run flutter run).

*Important: The supplied record code only shows the title, Body Text and a floating action button. It does not implement BMI inputs or calculation, so it can demonstrate a Flutter screen but not a working BMI calculator.*

## Experiment 11 – Real-Time On-Device Object Detection Using Flutter and TensorFlow Lite

**How to run (Flutter + TFLite):**

1. Install Flutter and verify it using flutter doctor.

2. Create the project: flutter create object_detection_app; then cd object_detection_app.

3. Add camera and tflite_flutter dependencies to pubspec.yaml and run flutter pub get.

4. Obtain a compatible SSD MobileNet .tflite model and its matching labels file. Place them in assets/models/ and register both assets in pubspec.yaml.

5. Add camera permission in AndroidManifest.xml. Apply the model's required input/output tensor shapes and image preprocessing.

6. Implement camera initialization, frame conversion, inference, confidence filtering and bounding-box rendering.

7. Run on a physical Android phone where camera access is available: flutter run.

*Important: The record contains only setup and partial model-loading/inference snippets. It does not provide the complete runnable camera app or the model/labels files. The code as supplied cannot produce live object detection by itself.*

## Quick setup commands

```
flutter doctor

flutter --version

flutter create bmical
```

```
cd bmical

flutter pub get

flutter run -d chrome
```